



Technology, property rights and organizational diversity in the software industry

Fabio Landini*

Department of Economics, University of Siena, Piazza San Francesco 7, 53100 Siena, Italy

ARTICLE INFO

Article history:

Received 14 March 2011

Received in revised form 13 January 2012

Accepted 14 January 2012

Available online 25 January 2012

JEL classification:

C73 (evolutionary games)

D23 (transaction costs

property rights)

L17 (open-source products and markets)

O34 (intellectual property rights)

Keywords:

Software production

System design

Institutional diversity

Transaction costs

Evolutionary games

ABSTRACT

Why do open- and closed-source productions co-exist? To address this question, the paper studies the viability of distinct systems for software development. The model shows that: (a) for low design costs of modularity, both open- and closed-source productions are viable systems; (b) closed-source production is more likely to be adopted the greater the expected rents on software; and (c) production efficiency is not a necessary condition for the stochastic stability of a system to obtain. These three results can shed light on the emergence of organizational diversity in the software industry. The paper adds to the literature in three ways: first, it considers property rights and technology as endogenous variables in the process of system design; second it argues that in producing software multiple equilibrium designs may exist; and third, it shows that, in because of high rents and low design costs of modularity, production inefficiency can be persistent.

© 2012 Elsevier B.V. All rights reserved.

1. Introduction

In the software industry there presently exist two distinct ways of organizing production. On one hand there exist closed-source production, which relies on exclusive copyright claims, hired workforce and hierarchical coordination. On the other open-source production, which combines non-exclusive copyright claims, (mainly) unpaid work and peering-based coordination. Although both forms of production have existed since long time in the software industry,¹ none of them seems to effectively prevail. On the contrary, open- and closed-source

productions tend to co-exist in most market segments.² This is clearly the case, for instance, in the market for web servers, where the competition between Apache (open-source) and Microsoft Web Server (closed-source) is more

software all the way through the present days. On this point see Lerner and Tirole (2002, 2005) and McGowan (2001).

² For detailed information on the trend of market shares within the distinct market segments see: Netcraft, web server survey, July 2010. URL <http://news.netcraft.com/archives/2010/>, last time checked: 29th of July 2010. NetApplications, Top browser share trend, 2010 <http://marketshare.hitslink.com/browser-market-share.aspx?qprid=1>, last time checked: 29th of July 2010. Gartner Inc., competitive landscape: mobile devices, 1q10, 2010. URL http://www.gartner.com/DisplayDocument?doc_cd=2009461.ref=g.rss, last time checked: 29th of July 2010. For data on both database and desktop operating systems see David A. Wheeler (2007), Why Open Source Software/Free Software (OSS/FS, FLOSS, or FOSS)? Look at the Numbers! URL <http://www.dwheeler.com/oss.fs.why.html>, last time checked: 16th of August 2010.

* Tel.: +39 0577 232661; fax: +39 0577 232620.

E-mail address: landini4@unisi.it

¹ The free sharing of software among programmers (mainly working for public institutions such as universities) was an established practice in the early 70s, and it has survived side by side to the production of proprietary

than 15 years long. A similar trend can be observed in the market for web browsers, where Internet Explorer (closed-source) has been steadily competing with Mozilla Firefox (open-source), and now also with Google Chrome (open-source). An almost equivalent situation exists in the market for database, with MySQL (open-source) competing against Oracle and SQL Server (closed-source), and in the market for mobile and desktop operating systems, with iPhone, RIM and Windows Mobile (closed-source) competing against Symbian and Android (open-source), and Microsoft Windows (closed-source) competing against GNU/Linux (open-source). Overall, looking at these as well as other market segments (e.g. office suites, finance and accountability packages, mail servers), it seems that for most part of the last two decades open- and closed-source software packages have simply co-existed in the market.

From the perspective of the standard organizational economics literature—e.g. Williamson (1985), the sustained co-existence of open- and closed-source productions is somewhat puzzling. According to this view, in fact, for any given set of available technologies, systems of production based on relatively inefficient allocations of property rights should be less likely to be adopted than more efficient ones, and thus a tendency towards organizational uniformity should emerge (at least for technologically equivalent products). In the software industry, on the contrary, we observe that for almost any kind of software package, two systems of production based on different property rights regimes co-exist. Why, then, do open- and closed-source productions co-exist? When both systems are effectively viable, what leads an organization to adopt one type of production rather than the other? Does this choice always entail that production efficiency is achieved? These are the main questions addressed in the paper.

The paper takes a system design approach to study software production. A system design is defined as a specific combination of two domains: technology and property rights. Both domains are assumed to be endogenous relative to each other, in the sense that technology is designed in order to adjust to the characteristics of property rights and viceversa.³ On this basis, the model focuses on the decision-making dynamics leading to the adoption of a particular system design, and not on actual production. Although such an approach is highly abstract, it offers a useful representation of the complementarities that may exist among distinct design domains. The model shows that: (a) for sufficiently low design costs of modularity, both open- and closed-source productions are viable systems for software development; (b) when both systems are viable, closed-source production is more likely to be adopted the greater the expected rents on software; and (c) production efficiency is not a necessary condition for the stochastic stability of a system to obtain.

These three results, together with the widespread diffusion of digital technologies in the mid '90s, may offer an

explanation for the exceptional degree of organizational diversity that we presently observe. The intuition is an analogy with the process of biological speciation. Acting as an exogenous shock, the diffusion of digital technologies has suddenly enlarged the set of software packages for which both open- and closed-source productions became viable systems. In consequence of the complementarities existing among distinct design domains, however, some organizations faced important constraints in the process of adapting their system to the new environment, and divergent trajectories of organizational speciation have emerged.⁴ Only the organizations that, in because of low expected rents, lose little by shifting to open-source production were in the position of actually making the move, while the others got locked into the old system. As a result open- and closed-source productions have increasingly co-existed.

In the literature on free/open-source software (FOSS) several works have investigated the viability of open-source production from both a theoretical and empirical point of view—see Benkler (2002), Johnson (2006), Baldwin and Clark (2006), David and Rullani (2008), von Hippel (2007) among the others. Significant emphasis, in particular, has been placed on the role and functioning of on-line communities of developers—see Lerner and Tirole (2002), David and Shapiro (2008), Lakhani and Wolf (2005), Lakhani and von Hippel (2003), den Besten et al. (2008) and Shah (2006). Less attention, however, has been paid on the effective co-existence of alternative systems for software development, and on the factors leading to the adoption of one system design when multiple options are available. This paper is explicitly aimed at filling such gap.

The paper is also related with the more general literature on the evolution of economic diversity (see Pagano and Nicita, 2001). The latter includes the evolutionary theories of the firm (Nelson and Winter, 1982) as well as the comparative studies on institutions and their emergence (Aoki, 2001, 1998). In this respect the paper presents one of the first attempt to apply the analysis of institutional diversity to the case of software production. For a similar approach see Baldwin and von Hippel (2011).

The structure of the paper is the following. Section 2 defines the concept of system design, and characterizes the domains of technology and property rights. Section 3 presents the model. Section 4 studies the asymptotic stability of the equilibrium designs. Section 5 transforms the decision-making dynamics defined in Section 4 into an ergodic process and studies stochastic stability. Section 6 discusses the model's results and uses them to explain the emergence of organizational diversity in the software industry.

2. Technology, property rights and system design

A simple definition of a system for software development can be based on two domains: the first is technology (T), i.e. the technological characteristics of the resources used in software production; the second is property rights

³ The system design approach adopted in this paper is inspired by the analysis of cooperative human systems design outlined by Benkler (2010). For a similar focus on a two-dimensional system defined in terms of technology and property rights see Pagano (1993).

⁴ On the concept of organizational speciation see Pagano (2001).

(R), i.e. the set of rights on the resources employed in the system (including software, human labor and hardware) and on the system itself. Depending on the way in which R and T combine, different systems for software development may exist.

In domain R , as suggested by Benkler (2002), software developers generally have two main options: an open-source regime $R^O \in R$, in which a marginal (or absent) use of employment contracts is combined with non-exclusive copyrights claims and decentralized hardware ownership; and a closed-source regime $R^C \in R$, in which a wide use of employment contracts is combined with exclusive copyrights claims and centralized hardware ownership. Both such regimes are widely used in the software industry, and tend to be associated with fairly different organizational structures: flat communities of self-selected peer producers in the case of R^O and managerial hierarchies based on hired labor in the case of R^C .

In domain T , the characterization of the alternative resources available to developers is far more complex. In such a domain, in fact, several variables ranging from procedural routines to physical equipments play a crucial role in determining how a piece of software is produced. A comprehensive representation of technology is thus difficult to obtain. For this reason, in this paper, I choose to focus on two variables only: (i) the degree of modularity in code architecture and (ii) the distribution of cognitive labor across software modules. The combination of these two variables, I believe, can offer a general enough characterization of technology, although at a cost in terms of exhaustiveness.⁵

Modularity is a well known concept in the literature on system design and refers to the way in which a system is decomposed (Simon, 1962). Baldwin and Clark (2000) describes a modular system as one in which the elements (e.g. tasks) are partitioned into subsets called modules. Within each module, elements of the system are densely dependent and interconnected. Across modules, elements are independent or nearly so. As suggested by Parnas (1972) modularity is a crucial property of a system since it facilitates coordination by reducing the amount of information that needs to be exchanged. At the same time, as argued by Langlois and Garzarelli (2008), modularity militates in favor of specialization, and thus it increases productivity.

Although authors tend to agree on the general principles underlying modularity, they vary in their definitions and measurements. In the context of this paper, and with particular reference to software production, I will define the degree of modularity as follows. Consider a piece of software as represented by set $S = \{s_1, s_2, \dots, s_m\}$ where $s_i \in S$ for $i = 1, \dots, m$ is the i -th task that is to be performed. Define a code design $\bar{s}$ as a set of dependences among the different tasks in S , and call $\bar{S} = \{\bar{s}_1, \bar{s}_2, \dots, \bar{s}_o\}$ the set of all possible designs. Define $|\bar{s}_j|$ for $j = 1, \dots, o$ as the

cardinality of $\bar{s}_j$, i.e. the number of dependences included in design j . Call $|\bar{s}|^{max} = \max\{|\bar{s}_1|, |\bar{s}_2|, \dots, |\bar{s}_o|\}$ the design with the greatest number of dependences. Then, the degree of code modularity associated with design j can be defined as $M^j = 1 - (|\bar{s}_j|/|\bar{s}|^{max})$. When $M^j = 1$ all tasks are independent and code modularity is maximum. On the contrary, when $M^j = 0$ design j corresponds to the one with the greatest number of dependences and modularity is minimum. Overall, the value of M^j tells how finely grained is the code architecture associated to design j .⁶

In addition to code modularity, the other main resource that I consider as a component of technology is cognitive labor. The latter, in particular, takes the form of problem solving activities and creative tinkering aimed at source code development. As suggested by Benkler (2006) cognitive human labor of this sort has by far become the most important input in the production of information goods, and software, in because of its complexity, represents a particularly relevant case in this category.

A simple way of accounting for cognitive labor is by considering the average units of work (say, hours) devoted to the development of each software module. Define a module as a collection of interdependent tasks. Given any design $\bar{s}_j$, call $P_j = \{p_1, p_2, \dots, p_v\}$ the set of modules associated with j . Define L_{jk} for $k = 1, \dots, v$ the hours of work devoted to the development of module k in j . Then, the average units of cognitive labor employed in design j can be defined as $\bar{L}^j = (\sum_k L_{jk})/|P_j|$ where $|P_j|$ is the cardinality of set P_j . For the sake of simplicity I will assume $\bar{L}^j$ to be normalized in such a way that it ranges from 0 to 1. When $\bar{L}^j = 1$ the average amount of cognitive labor employed in the development of design j is maximum, i.e. equal to the maximum amount of cognitive labor effectively (or legally) available for each module. On the contrary, when $\bar{L}^j = 0$ such an amount is minimum. Notice that $\bar{L}^j = 0$ does not mean that no cognitive labor is used at all, but rather that the average amount of work devoted to the development of each software module is so small that it is almost negligible.

This definition of M and L allows one to treat such variables as two factors of production in the standard economic sense. Both M and L , in fact, positively contribute to production, and can to a certain extent be considered substitute of each other. For a given size of software S , in fact, an increase (decrease) in M tends to decrease (increase) the average size of the software modules and therefore reduces (augments) the amount of cognitive labor L needed to develop each module. Under this interpretation the nature of a generic technology $\bar{T}$ can be defined by the factor proportion (or intensity) $\bar{T}^j = M^j/\bar{L}^j$. A technology $\bar{T}$ is defined as relatively modular when compared to a technology $\bar{T}^h$ if and only if $\bar{T}^j > \bar{T}^h$. The idea is that while in a modular technology highly modular software (high M^j) is developed by programmers devoting relatively small portions

⁵ For studies that place similar emphasis on modularity and cognitive labor while studying the nature of the resources employed in software production see Baldwin and Clark (2006), West and O'Mahony (2008) and Langlois and Garzarelli (2008).

⁶ This way of measuring modularity is a simplified version of the one employed in Mac Cormack et al. (2006). In both cases modularity is evaluated by looking at the pattern of dependences in the overall software architecture. Also in Mac Cormack et al. (2006) the degree of modularity associated with a given code architecture can be evaluated only in relative terms.

of time to programming (low L^l), in a non-modular technology weakly modular software (low M^h) is developed by programmers devoting large portion of time to programming (high L^h). In this framework, the nature of the choice that is open to developers in domain T can be reduced to set the factor proportion M/L .

Given this characterization of domains R and T , the remaining sections of the paper will study the viability and persistence of systems for software development based on different combinations of R and T . In particular, the attention will be placed on the factors that make one system design more or less likely to be adopted as compared to another, and on the existing link between systemic efficiency and actual adoption.

3. The model

3.1. General setting and assumptions

The type of interaction that I model in this paper is one in which a group of agents (possibly a group of programmers) projects to develop a new piece of software. In order to do so, and before production can actually take place, they need to choose how to design the system of production, i.e. they need to set a specific combination of R and T . Once the system is designed, production occurs and the agents earn the associated payoffs. The model is based on six distinctive assumptions, that I believe are of broad empirical relevance.

First, both property rights and technology are treated as endogenous variable in the model, i.e. both R and T are choice variables in the system. In particular, I assume that R is set in order to maximize expected profits given T , and T is set in order to maximize expected profits given R . While the first causality is a standard assumption in organizational economics – see for instance Williamson (1985) – the one running from property rights to technology is less common. Some of the early authors making a point in favor of this type of causality were Marglin (1974), Putterman (1982) and Bowles (1985), and lately it has been defended also by Pagano and Rowthorn (1994), Pagano and Rossi (2004) and Pagano (2011). The main argument that is put forward by these authors is that, similarly to the case in which property rights are allocated so as to minimize the transaction costs associated with a given technology, also technology may be designed so as to optimally adjust to given rights. Although this argument has received some critiques in the literature (Williamson, 1985), it is very reasonable when applied to the case of digital productions, such as software. As suggested by Elkin-Loren and Salzberger (2000), in fact, in this type of productions technology is so flexible and cheap to adapt that its design is likely to be endogenous to the nature of legal relations. In this respect, there are several examples of software packages in which the code architecture has been expressly modified in order to fit with a given allocation of property rights.⁷

Second, I assume that technology is not only flexible, i.e. cheap to modify—but also malleable. In particular, I assume M and L to be perfect substitute, i.e. the very same piece of software can be produced by substituting each fraction of M with a fraction of L and *vice versa*. This in turn implies that the structure of S is so malleable that there always exist a design $\bar{s}$ such that M can be increased and/or reduced at wish, and that an increase (decrease) in M is always associated with a reduction (increment) in the average size of the software modules. Moreover, perfect substitutability implies that, below the maximum amount available, there are no (physical or legal) constraints in the amount L that can be employed in each module. These implications are fairly stringent, but are reasonable so long as digital production is considered.

Third, I assume that software gives rise to two main types of return. The first is the expected rents on the sale of the software package, which I call $z(M, L, R)$. Since such rents exist only under R^C , I assume $z(M, L, R)$ to take the following form:

$$z(M, L, R) = \begin{cases} 0, & \text{if } R = R^O \\ z(M, L), & \text{if } R = R^C \end{cases} \quad (1)$$

where $\partial z/\partial M > 0$ and $\partial z/\partial L > 0$. The second type of return is instead associated with any other kind of expected return that can be earned from software apart from rents, including the sale of services, advertisements and network effects. This type of return, which exist also under R^O , is captured by a function $Q(M, L)$ such that $\partial Q/\partial M > 0$ and $\partial Q/\partial L > 0$. The shape of this function is assumed to be independent of the property rights regime. Hence, in line with most of the authors in the literature, I model R^C and R^O as being associated with two distinct revenue generation models: one that gives rise to $z(M, L, R) + Q(M, L)$ in the first case, and another that generates only $Q(M, L)$ in the second. As we will see, this distinction will play a crucial role in the model.

Fourth, I assume that different property rights regimes entail different costs. In particular, I consider two main types of costs: design costs and transaction costs.⁸ On the side of design costs, I call d the design cost of modularity, i.e. the cost of modularizing the code architecture. On the side of transaction costs, I call l the transaction cost of labor, i.e. the cost of inducing actual effort from labor; and m the transaction cost of skills allocation, i.e. the information cost associated with the allocation of cognitive skills across software modules. The latter, in particular, can be interpreted as the information cost that is incurred in screening the skills of individual programmers first, and then in assigning programmers to the development of specific modules within the system. Since skills are costly to evaluate (both subjectively and from a third-party), the value of m can be relatively high, and tends to increase in the level of M . The

⁷ A good example of this causality comes from the common practice of refactoring the source code before going open-source. Code refactoring, i.e. the process of changing the non-functional attributes of a software in

order to foster the source code's maintainability and extensibility, indeed entails an improved readability and modularity of the code architecture. The author is grateful to Yochai Benkler for pointing out the analogy between code refactoring and the causality running from property rights to technology.

⁸ For a similar approach see Baldwin and von Hippel (2011).

reason is that the higher (lower) M , the more (less) modules exist in the system, the greater (smaller) the division of labor, and thus the more (less) complex and costly to evaluate the set of all possible allocations of programmers to modules.⁹ At the same time, I assume l to be monotonically increasing in L , and d to be monotonically increasing in M .¹⁰

Given this definition of the costs included in the model, I assume that each property rights regime R^O and R^C is characterized by a different transaction cost advantage. Under R^O the allocation of cognitive skills is not hierarchically determined, but rather relies on the self-identification of programmers into the modules they wish to contribute. For this reason, as suggested by Benkler (2002), under R^O there tends to be a cost advantage in terms of m as compared to R^C , because neither screening nor allocation of individual skills is to be undertaken on the side of the organization.¹¹ I will call such cost advantage $x(<m)$. At the same time, as partly suggested by David and Rullani (2008), the fact that in a system based on R^C most of the programmers are hired rather volunteers, makes it easier for such a system to mobilize labor as compared to a system based on R^O . For this reason I assume that under R^C there is a cost advantage in terms of l . I will call the latter $y(<l)$. On this basis, I write the transaction costs function as follows

$$C(M, L, R) = \begin{cases} (m - x)M + lL, & \text{if } R = R^O \\ mM + (l - y)L, & \text{if } R = R^C \end{cases} \quad (2)$$

where $m > l - y$ and $l > m - x$, the latter conditions meaning that there exist transaction cost advantages in the use of different factors of production not only between but also within production systems. In addition to this, I write the total design cost of modularity as dM . Such a cost, however, is assumed to be the same under both property rights regimes.

⁹ A high level of M has also the positive effect of reducing the average size of each module. This in turn increases the probability to find a good matching between the skills of programmers, and the set of tasks that are to be developed in any given module. Such a positive effect tends to hold under both property right regimes, and in the model is captured by the fact that $\partial Q/\partial M > 0$. At the same time, however, the fact that modules are small does not imply that the number of individual skills that need to be evaluated is small too. Even when modules are as small as a single task, an optimal allocation of programmers to modules still requires a detailed mapping of individual skills, including inter- and intra-individual differences. Only on the basis of this information, in fact, the organization is able to balance the skills and talents of each single programmer, and thus guarantee an optimal allocation of skills within the system.

¹⁰ Surely there exist other transaction costs that can be associated with both labor and modularity. With respect to the latter, in particular, the coordination costs among programmers certainly play an important role (Parnas, 1972). For the sake of traceability, however, I choose to focus only on the transaction costs whose value is likely to change according to the type of property rights regime. This is not the case, for instance, if we consider coordination costs, which have instead a structural (or architecture-based) nature.

¹¹ Also under R^O there is still need of some skills comparison among peers, to decide who does what within the system. With respect to a hierarchical organization, however, each programmer can at least save on the costs of screening her own skills. Moreover, I assume that each member of the peer group faces a cost in acquiring information about the skills of others which is not greater (and possibly lower) than the one of a manager in a standard firm.

The fifth assumption that I consider is that, independently of the system which is effectively designed, the necessary conditions for production to take place are satisfied. This in turn implies that there exist a market and a demand for the software package, as well as an adequate factors endowment, especially in terms of L . When R^O is chosen in domain R the latter condition amounts to assume that there exist a community of developers willing to volunteer in writing the source code (remember that no employment contract is used under R^O). The model abstracts from both the way in which such community is gathered and the way in which the community works. Previous studies on participation in FOSS communities have successfully dealt with both these issues (Lerner and Tirole, 2002; David and Shapiro, 2008; Lakhani and Wolf, 2005; Lakhani and von Hippel, 2003; den Besten et al., 2008; Shah, 2006; von Hippel and von Krogh, 2003; Gambardella and Hall, 2006; Bonaccorsi and Rossi, 2003; David and Rullani, 2008), hence I take them as given in the model. On the contrary, I mainly focus on the dynamics of system design.

Finally, I assume that agents have limited cognition and come to a choice through a process of adaptive learning. This does not mean to disregard the intention of each agent to maximize his own payoffs. As we will see such an intention plays a crucial role in the model. By 'limited cognition' I simply mean the limited capacity and predisposition to engage in extraordinary complex and costly cognitive exercises, such as making advanced inferences about what others will do and about the way the world works. Several contributions in both cognitive psychology and behavioral economics provide empirical evidence in support of this assumption—e.g. Kahneman (2003).

3.2. A simple game of system design

Given the above assumptions, a simple model of system design can be defined as follows. Let us consider an economy in which a voluntary association of economic agents projects to develop a piece of software. We could think of it as a standard organization such as a software house, or more generally a group of programmers, involved in the development of a new software package. In such an economy, in order for this package to be produced, a specific system for software development has to be designed.

The organization is composed of two distinct groups of agents: $n_r(>0)$ identical 'financiers' (r), who own the organization and are responsible for selecting the property rights regime R ; and $n_t(>0)$ identical 'production managers' (t) who are members of the organization and are responsible for the design of technology T . Agents in both groups make their choices in order to maximize individual payoffs, which in turn depend on the expected returns and costs of software production.¹²

As described in the previous section software is assumed to give rise to two types of expected return: the rents on software sales $z(M, L, R)$ and the returns other than rents, indicated by $Q(M, L)$. In order to account for

¹² For a similar distinction between 'financiers' and 'production managers' see Pagano (2011).

Table 1

Strategic game of system design, matrix of payoffs. Note: each cell of the matrix corresponds to a different system for software development; the reported payoffs are the ones that the agents get if production takes place in each alternative system.

Copyright (r)	Technology (t)	
	Modular, T^M	Non-modular, T^L
Open-source, R^O	$\frac{Q-(m-x)}{n}, \frac{Q-(m-x)}{n} - \frac{d}{n_t}$	$\frac{Q-l}{n}, \frac{Q-l}{n}$
Closed-source, R^C	$\frac{Q-m}{n} + \frac{z}{n_r}, \frac{Q-m}{n} - \frac{d}{n_t}$	$\frac{Q-(l-y)}{n} + \frac{z}{n_r}, \frac{Q-(l-y)}{n}$

asymmetric relations within the organization, I assume the former to be appropriated in equal shares only by the members of group r (r -agents), while the latter to be earned in equal shares by the members of both groups. Similarly, the production of software is assumed to generate two types of costs: transaction costs m and l , which are sustained by all members of the organization; and design costs d , which are sustained only by the agents involved in the modularization of code architecture, i.e. the members of group t (t -agents). On this basis, given Eqs. (1) and (2), the payoffs of r - and t -agents can be written as follows:

$$\pi_r(R, T(M, L)) = \frac{z(R, L)}{n_r} + \frac{[Q(M, L) - C(M, L, R)]}{n} \quad (3)$$

$$\pi_t(R, T(M, L)) = \frac{[Q(M, L) - C(M, L, R)]}{n} - \frac{dM}{n_t} \quad (4)$$

where π_i (for $i = r, t$) is the payoff of an i -agent and $n = n_r + n_t$ is the size of the organization. Given Eqs. (3) and (4), the choice of the agents takes the following form: r -agents choose that property rights regime that maximizes ownership rents given the existing technology, i.e. they choose $R = \arg \max \pi_r(R, T(M, L))$, taking as given the combination of M and L ; while t -agents choose that technology that maximizes profits given the existing property rights regime, i.e. they choose $(M, L) = \arg \max \pi_t(R, T(M, L))$, taking as given the value of R . For the sake of simplicity, and without loss of generality, I assume that in both domains R and T the choice set Σ_i (for $i = r, t$) consists of only two alternatives. In domain R , r -agents can choose between an open-source regime R^O and a closed-source regime R^C , i.e. $\Sigma_r = \{R^O, R^C\}$. In domain T , t -agents can choose between a modularity-intensive technology $T^M = M^M/L^M$ and a labor-intensive technology $T^L = M^L/L^L$, i.e. $\Sigma_t = \{T^M, T^L\}$. To simplify the analysis, given the assumption of perfect substitutability between M and L , I assume $M^M = 1, L^M = 0, M^L = 0$ and $L^L = 1$, which in turn implies $\infty = T^M > T^L = 0$. Moreover, I also assume $Q(1, 0) = Q(0, 1) = Q > 0, z(1, 0, R^C) = z(0, 1, R^C) = z > 0$, and $m = l$.

Since agents are assumed to be identical within the same group, we can now consider two representative agents r and t . On this ground, a strategic game of system design can be defined by the triplet $\Gamma = (I, \Sigma, \pi)$ where $I = \{r, t\}$ is the set of players, $\Sigma = \Sigma_r \times \Sigma_t$ is the set of strategy profiles and $\pi = \{\pi_r(\sigma), \pi_t(\sigma)\}$ for $\sigma \in \Sigma$ is the vector function of players' payoffs, where $\pi_r(\sigma)$ and $\pi_t(\sigma)$ are given by Eqs. (3) and (4), respectively. Table 1 reports a normal-form representation of Γ . With respect to the latter I introduce the following definitions:

Definition 1. A system design in game Γ corresponds to a pure strategy profile $\sigma = \{\sigma_r, \sigma_t\} \in \times_{i \in I} \Sigma_i$, where $\sigma_r \in \Sigma_r$ and $\sigma_t \in \Sigma_t$ is the pure strategy adopted by player r and t , respectively.

Definition 2. A system design $\sigma^* = \{\sigma_r^*, \sigma_t^*\}$ is an equilibrium design, if the correspondent pure strategy profile is a Nash equilibrium of game Γ .¹³

Game Γ is characterized by four distinct system designs: $\{R^O, T^M\}$, $\{R^C, T^M\}$, $\{R^O, T^L\}$ and $\{R^C, T^L\}$. I now determine the conditions under which each of the four system designs is an equilibrium design.

Proposition 1. Suppose $Q > 0, z > 0, l = m > 0, x > 0$ and $y > 0$. Then: (a) For any $d > 0$, $\{R^C, T^L\}$ is always an equilibrium design; (b) For any $d > 0$, $\{R^C, T^M\}$ and $\{R^O, T^L\}$ are never equilibrium designs; (c) If the transaction cost advantage under R^O is sufficiently large, i.e. $x \geq z/\delta$ where $\delta = n_r/n$, and the design cost of modularity d is sufficiently small, i.e. $d \leq x\xi$ where $\xi = n_t/n$, then $\{R^O, T^M\}$ is an equilibrium design.

Proof. See Appendix A. $\square$

When point (c) in Proposition 1 holds, game Γ is affected by strategic complementarities and multiple equilibrium designs exist. In the latter case, it is interesting to characterize the degree of efficiency of each such design. Let's consider the following:

Definition 3. Take any system design $\sigma^j = \{\sigma_r^j, \sigma_t^j\} \in \times_{i \in I} \Sigma_i$. The degree of efficiency ψ^j is defined as the total joint surplus generated by design j , i.e. $\psi^j = \pi_r(\sigma^j)n_r + \pi_t(\sigma^j)n_t$.

Taking equilibrium $\{R^C, T^L\}$ as the benchmark (let's call it design 0), the efficiency properties of $\{R^O, T^M\}$ (let's call it design 1) can be thus defined as follows:

Proposition 2. Suppose that multiple equilibrium designs exist. Then: (a) $\{R^O, T^M\}$ is relatively efficient (i.e. $\psi^1 > \psi^0$) if and only if $x \geq y + d$ and $z < x - y - d$; (b) $\{R^O, T^M\}$ is Pareto superior if and only if $x > y, z < \delta(x - y)$ and $d < \xi(x - y)$.

Proof. See Appendix A. $\square$

Proposition (1) and (2) point towards the existence of an interesting relationship between technology, property rights and the exogenous factors influencing the environment in which software production takes place. When both z and d are high, because for instance competition in the industry is low and/or the process of code modularization is subject to some fixed rigidities, $\{R^C, T^L\}$ is the only equilibrium design and it is also efficient. This can reflect situations in which standard firm-based productions is the predominant way of developing software, e.g. in the early years of the software industry when digital technologies were not as fully developed as today and the number of firms operating in the market was limited. On the contrary, when both z and d are relatively small compared to x , the game is characterized by multiple equilibria. In this case, alongside $\{R^C, T^L\}$, also $\{R^O, T^M\}$ qualifies as an equilibrium

¹³ According to this definition an equilibrium design represents what Pagano and Rowthorn (1994) call an organizational equilibria.

design and under certain conditions it turns out to be relatively efficient. The intuition is that if x is sufficiently large – because for instance the self-identification of programmers into modules is particularly effective – the reduction in transaction costs associated with R^O more than compensate both the loss in rents that affect r -agents when R^O is chosen in place of R^C , and the greater cost which is incurred by t -agents when the code architecture has to be modularized. Thanks to the combination of these two effects, $\{R^O, T^M\}$ is able to generate a greater surplus than $\{R^C, T^L\}$.

The cases in which both $\{R^O, T^M\}$ and $\{R^C, T^L\}$ are equilibrium designs reflect situations in which, for the same software package, both open- and closed-source productions are viable. The main difference between these two productions, beside the adoption of a different property rights regime, is in the design of technology, with open-source production adopting a relatively more modular technology than closed-source production. Some empirical evidence in support of these distinct property rights-technology combinations comes from Mac Cormack et al. (2006). In this sense, the above game provides a relatively simple micro-foundation of their results.

Viability, however, is not a synonym of actual adoption. When multiple equilibrium designs are viable, in fact, an organization still has an incentive to adopt the solution that ensures the greatest economic return, with the consequence that some viable options may not be even tried (or if they are tried, they are readily abandoned). What remains to be investigated, therefore, are the factors that may lead an organization to adopt one specific design as opposed to another when both options are available—i.e. to play one particular Nash equilibrium in the above game, and that may eventually cause similar organizations to diverge in their choices. In order to understand that, a model of the decision-making dynamics leading to the actual adoption of a system design has to be introduced.

4. Dynamics

To provide a framework for understanding the process through which a particular system design is adopted, I now study the asymptotic stability of the two equilibria $\{R^O, T^M\}$ and $\{R^C, T^L\}$. In order to do so, I explicitly model the governance process that leads r - and t -agents to come to a final choice, starting from the assumption that two equilibrium designs exist. Such a governance process takes the form of a decision-making dynamics through which the design of domains R and T co-evolve, before production can actually take place. Instead of assuming an exogenous law of motion, I model the dynamics by adopting the now-standard evolutionary game theoretic approach (Gintis, 2009). This approach has two main advantages: first, it easily allows one to extend the previous static game into a dynamic game; second, it micro-founds the evolution of decision-making on credible behavioral rules, including adaptive learning and limited cognition. This obviously comes at a cost, which is mainly due to the high degree of abstraction in modeling the interaction. In the context of this paper, such an abstraction takes the form of a repeated voting mechanism thanks to which the agents in the two groups can express their preferred design choice.

The dynamics of voting is modeled as follows. At the beginning of every time period τ agents in both groups express an individual and autonomous vote in their own domain of choice, i.e. r -agents vote in domain R having $\Sigma_r = \{R^O, R^C\}$ as choice set, and t -agents vote in domain T having $\Sigma_t = \{T^M, T^L\}$ as choice set. Let

$$\rho_\tau = \frac{n_\tau^O}{n_\tau^O + n_\tau^C} \quad \omega_\tau = \frac{n_\tau^M}{n_\tau^M + n_\tau^L} \quad (5)$$

be the fraction of agents voting R^O and T^M , respectively, with n_τ^j being the number of agents voting for option j at time τ and $n_r = n_\tau^O + n_\tau^C$ as well as $n_t = n_\tau^M + n_\tau^L$ for all $\tau = 1, \dots, \infty$. At τ_0 these votes are exogenously determined by causes not expressly modeled (it can be due to preferences or previous experience). Then, for any $\tau > \tau_0$, each agent updates her vote following the updating process described below. Once votes are expressed, agents are paired across the two groups to compare their individual decisions. To lend some concreteness to the model we can imagine such pairings as periodic meetings between ‘financiers’ and ‘production managers’ to discuss the costs and benefits of distinct system design. In the course of such meetings each agent receives two pieces of information. The first one is the distribution of votes in each group. The second one consists of the payoffs reported in Table 1—i.e. the payoffs that each agent would receive if the software were to be developed under each system design. Such payoffs, however, do not immediately translate in individual earnings since production takes place only when “near” unanimity obtains, where the latter means that the following conditions hold: $\rho_\tau < \varepsilon$ or $\rho_\tau > 1 - \varepsilon$ and $\omega_\tau < \varepsilon$ or $\omega_\tau > 1 - \varepsilon$ with $\varepsilon > 0$ being arbitrarily small, i.e. only a negligible portion of agents disagrees with the majority in both groups. Once production takes place, agents earn the payoffs associated with the system design chosen by the majority, i.e. version 1.0 of the software package comes out. In this framework the “near” unanimity condition can be simply interpreted as a managerial rule which ensures that consistent practices are maintained within the organization. Once production has taken place, the process of vote updating starts anew with the initial distribution of votes being the one associated with the organization of production actually adopted. For the sake of simplicity I assume that agents have a time discount factor equal 1.

The process of vote updating takes the following form. At the beginning of every time period $\tau > \tau_0$ each agent in a group uses the two pieces of information obtained during the meeting occurred in the previous period in order to compute her expected final payoffs, where expectations are formed on the basis of the distribution of votes in the other group. Using the payoffs in Table 1 the expected payoffs to t -agents voting T^M and T^L can be written as

$$V_\tau^M(\rho_{\tau-1}) = \rho_{\tau-1} \left[\frac{Q - (m - x)}{n} - \frac{d}{n_t} \right] + (1 - \rho_{\tau-1}) \left[\frac{Q - m}{n} - \frac{d}{n_t} \right] \quad (6)$$

$$V_\tau^L(\rho_{\tau-1}) = \rho_{\tau-1} \frac{Q - l}{n} + (1 - \rho_{\tau-1}) \left[\frac{Q - (l - y)}{n} \right] \quad (7)$$

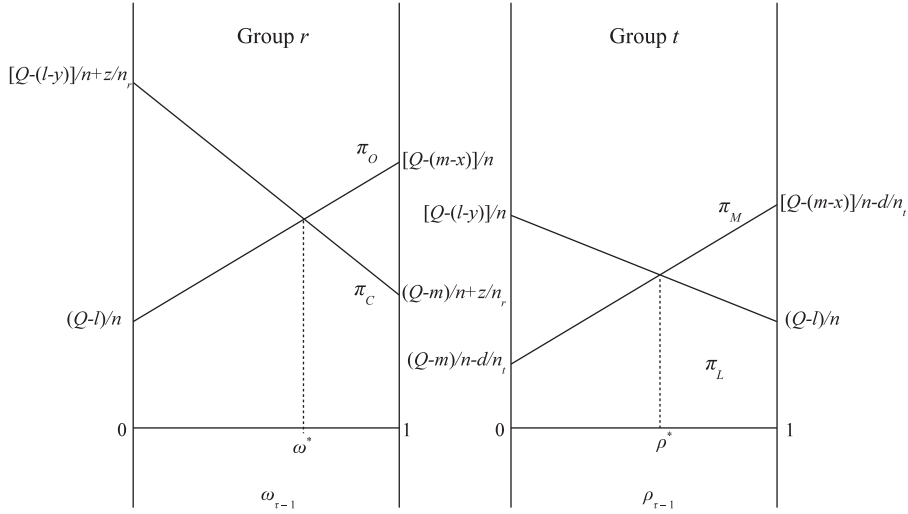


Fig. 1. Expected payoffs to agents from sub-groups r and t . Note that $\omega_{\tau-1}$ and $\rho_{\tau-1}$ are respectively the fractions of agent voting T^M and R^O in the previous period.

Similarly, the expected payoffs to r -agents voting R^O and R^C are respectively

$$V_{\tau}^O(\omega_{\tau-1}) = \omega_{\tau-1} \left[\frac{Q - (m - x)}{n} \right] + (1 - \omega_{\tau-1}) \frac{(Q - l)}{n} \quad (8)$$

$$V_{\tau}^C(\omega_{\tau-1}) = \omega_{\tau-1} \left[\frac{(Q - m)}{n} + \frac{z}{n_r} \right] + (1 - \omega_{\tau-1}) \left\{ \left[\frac{Q - (l - y)}{n} \right] + \frac{z}{n_r} \right\} \quad (9)$$

These expected payoff functions are reported in Fig. 1, assuming $x > y$. Once such functions have been computed and before choosing on a new vote, each agent meets with another agent randomly selected from her sub-group and compares the respective decisions. For instance, an agent a in group t has the opportunity to observe the vote expressed by another t -agent, named b , and to know her expected payoffs with a probability $\gamma d\tau$. If b expressed the same vote as a , a does not update. But if b expressed a different vote, a compares the two expected payoffs and, if b has a greater expected payoffs, switches to b 's vote with a probability equal to $\beta > 0$ times the payoff difference, retaining her own vote otherwise. While this updating process is not very sophisticated, it may realistically reflect individual cognitive capacities and it ensures that the standard economic assumption of utility maximization is preserved—i.e. each agent votes for the outcome that, given the distribution of votes in the other group, maximizes her expected payoffs.

Specifically, writing the probability that an agent who voted option j switches to option k at time τ as α_{τ}^{jk} we have:

$$\alpha_{\tau}^{jk} = \begin{cases} \beta(V_{\tau}^k - V_{\tau}^j), & \text{if } V_{\tau}^k > V_{\tau}^j \\ 0, & \text{if } V_{\tau}^k \leq V_{\tau}^j \end{cases} \quad (10)$$

for $j, k = T^M, T^L$ and $j \neq k$ in the case of t -agents and $j, k = R^O, R^C$ and $j \neq k$ in the case of r -agents. On this basis

the expected fraction of r -agents who votes R^O (R^O -voters) in period $\tau + d\tau$ is given by:

$$\rho_{\tau+d\tau} = \rho_{\tau} - \rho_{\tau}(1 - \rho_{\tau})\gamma d\tau \eta^C \beta (V_{\tau}^C - V_{\tau}^O) + (1 - \rho_{\tau})\rho_{\tau}\gamma d\tau \eta^O \beta (V_{\tau}^O - V_{\tau}^C) \quad (11)$$

where η^C and η^O are two binary functions such that $\eta^C = 1$ if $V_{\tau}^C > V_{\tau}^O$ and is zero otherwise, $\eta^O = 1$ if $V_{\tau}^O > V_{\tau}^C$ and is zero otherwise, and $\eta^C + \eta^O = 1$. Eq. (11) reads as follows: the expected fraction of R^O -voters at $\tau + d\tau$ is given by the fraction of R^O -voters at τ (first term), minus the fraction of R^O -voters who are paired with a R^C -voter and switch their type (second term), plus the fraction of R^C -voters who are paired with a R^O -voter and switch their type (third term). Similarly, the expected fraction of agents who votes T^M (T^M -voters) in period $\tau + d\tau$ is given by

$$\omega_{\tau+d\tau} = \omega_{\tau} - \omega_{\tau}(1 - \omega_{\tau})\gamma d\tau \eta^L \beta (V_{\tau}^L - V_{\tau}^M) + (1 - \omega_{\tau})\omega_{\tau}\gamma d\tau \eta^M \beta (V_{\tau}^M - V_{\tau}^L) \quad (12)$$

where $\eta^L = 1$ if $V_{\tau}^L > V_{\tau}^M$ and is zero otherwise, $\eta^M = 1$ if $V_{\tau}^M > V_{\tau}^L$ and is zero otherwise, and $\eta^L + \eta^M = 1$. Subtracting ρ_{τ} and ω_{τ} from both sides of Eqs. (11) and (12), respectively, dividing both equations by $d\tau$, and taking the limit as $d\tau \rightarrow 0$, we get:

$$\dot{\rho}_{\tau} = \rho_{\tau}(1 - \rho_{\tau})(V_{\tau}^O(\omega_{\tau}) - V_{\tau}^C(\omega_{\tau})) \quad (13)$$

$$\dot{\omega}_{\tau} = \omega_{\tau}(1 - \omega_{\tau})(V_{\tau}^M(\rho_{\tau}) - V_{\tau}^L(\rho_{\tau})) \quad (14)$$

where for the sake of simplicity, I assume $\gamma\beta = 1$. Eqs. (13) and (14) represent a system of differential equations which describes how the distribution of votes $\{\rho_{\tau}, \omega_{\tau}\}$ evolves over time. Given this dynamics, we are mainly interested in the stationary states, namely the states for which $\dot{\rho}_{\tau} = 0$ and $\dot{\omega}_{\tau} = 0$. Such states qualify as fixed-points of the dynamical system, and as voting equilibria of the organization.

Proposition 3. The dynamical system composed of Eqs. (13) and (14) is characterized by five voting equilibria: $\{0, 0\}$, $\{0,$

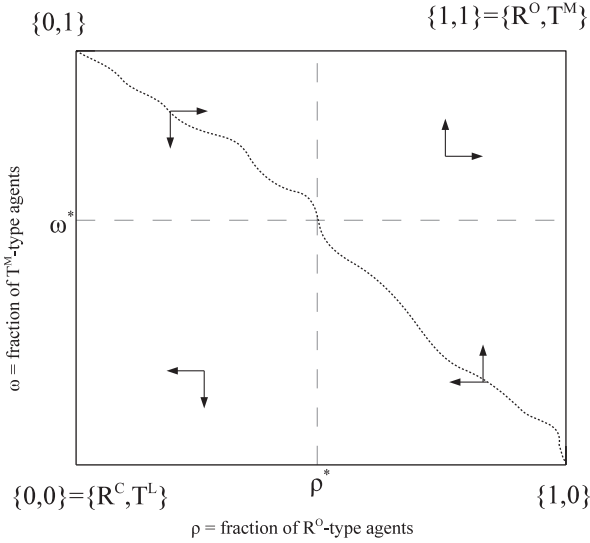


Fig. 2. Asymptotically stable states and out-of-equilibrium dynamics. Note: the arrows represent the disequilibrium adjustment in the number of voting agents (horizontal movements for the property rights domain and vertical for the technology domain).

$\{1, \{1, 0\}, \{1, 1\}$ and $\{\rho^*, \omega^*\}$, where $\rho^* = (y + d/\xi)/(x + y)$ and $\omega^* = (y + z/\delta)/(x + y)$. Out of these five equilibria, only two are asymptotically stable, namely $\{0, 0\}$ and $\{1, 1\}$; equilibrium $\{\rho^*, \omega^*\}$ is a saddle, whereas equilibria $\{0, 1\}$ and $\{1, 0\}$ are unstable.

Proof. See Appendix A. $\square$

Fig. 2 offers a graphical representation of the dynamical system composed of Eqs. (13) and (14). The arrows indicate the out-of-equilibrium adjustment. For a distribution of votes such that $\rho_t < \rho^*$ and $\omega_t < \omega^*$ (i.e. in the southwest region of Fig. 2), $\Delta\rho_t$ and $\Delta\omega_t$ are both negative and the organization will move towards the $\{0, 0\}$ equilibrium. This equilibrium corresponds to closed-source production where all r - and t -agents vote for R^C and T^L , respectively; I will call the latter equilibrium 0 (E_0). As reported in Proposition 3, such an equilibrium qualifies as an asymptotically stable state of the dynamics, meaning that when the organization is at this equilibrium it will never leave. Analogous reasoning holds for the northeast region of Fig. 2, where the organization converges towards the $\{1, 1\}$ equilibrium. In this case the asymptotically stable state corresponds to open-source production where r - and t -agents vote for R^O and T^M , respectively; I will call the latter equilibrium 1 (E_1). In the remaining regions of the state space, namely northwest and southeast, we may define a locus of states from which the system will transit to the interior equilibrium $\{\rho^*, \omega^*\}$ (the dashed line in Fig. 2). Such a state is stationary, but is a saddle: small movements away from ρ^* and ω^* are not self-correcting. For all the states below the locus transiting to $\{\rho^*, \omega^*\}$ the system will move to E_0 . On the contrary, for all the states above that locus the system will move to E_1 . As a consequence, there is no set of points for which the system converges to the equilibria $\{0, 1\}$ and $\{1, 0\}$. Such voting equilibria qualify as unstable states of the dynamics and they will never be reached.

5. Stochastic process

Under the present formulation, there being more than one absorbing state, the dynamic process modeled in Section 4 is non-ergodic, i.e. its long-run average behavior depends on initial conditions. If the organization starts with a distribution of votes in the area below the dashed downward-sloping line in Fig. 2—i.e. in the basin of attraction of E_0 , the organization will adopt design $\{R^C, T^L\}$ and thus closed-source production. On the contrary, if the initial state is a point in the area above the dashed line—i.e. in the basin of attraction of E_1 , the organization will adopt design $\{R^O, T^M\}$ and thus open-source production. In this framework, for any exogenously given starting point, the likelihood that any system design is actually adopted is greater the larger the basin of attraction of its correspondent voting equilibrium, whereas the following holds:

Remark 1. $\partial\rho^*/\partial x < 0$, $\partial\omega^*/\partial x < 0$, $\partial\rho^*/\partial d > 0$ and $\partial\omega^*/\partial z > 0$ imply that the basin of attraction of E_1 (E_0) is larger (smaller) the greater (smaller) x and the smaller (greater) d and z .

Such dynamics may certainly offer some useful insights on the process of system design, especially in highlighting the role that conventions and routines may play as sources of path dependency. In the model of Section 4, in fact, we assumed that the initial distribution of votes was exogenously given. The latter, however, may clearly be a function of what the organization did in the past, and in particular of the type of systems the organization has learned to design. If most of the software packages produced in the past were developed through design $\{R^C, T^L\}$, for instance, it is reasonable to expect most of the agents in both groups to have evolved a set of routines according to which such system is the conventional way of producing software within the organization. By relying on such routines, the agents may be thus inclined to vote for $\{R^C, T^L\}$ also in this case. The result is that, for any new package that is to be developed, the distribution of votes within this organization is highly likely to start in the southwest region of Fig. 2, and closed-source production is likely to be adopted. This sort of path dependency in the process of organizational design plays a crucial role in the evolutionary theories of the firm—see for instance Nelson and Winter (1982), and is surely of crucial importance in the field of software development.

At the same time, however, the non-ergodic nature of the above dynamics limits the possibility of explaining the reason why a specific path of system design is embarked in the first place. This can be an important limit, especially if we wish to explain the existence of organizational diversity in newly emerging market segments, where start-up companies play an important role. In order to avoid such limitation we need to transform the dynamical system into an ergodic process. Following Young (1998) and in particular Bowles (2006) and Naidu et al. (2010), I do so by introducing the possibility that agents engage in intentional idiosyncratic plays.

Suppose that every period there is a probability $\lambda \in (0, 1)$ that each agent is selected to undertake an intentional non-best response meant at explicitly influencing the voting process in favor of the system design that she prefers,

i.e. the one that ensures the greatest individual payoffs. The idiosyncratic plays accounting for non-best responses need not be irrational; it can be simply the result of experimentation. These stochastic events transform the dynamics into an ergodic process.

If one system design ensures to all agents in both groups payoffs greater than the other—i.e. $\{R^C, T^L\}$ and $\{R^O, T^M\}$ can be Pareto ranked, once the voting equilibrium associated with that system is reached, no idiosyncratic play occurs and the organization will remain in that state forever. In line with the mainstream approach to organizational evolution, therefore, highly efficient and conflictless designs tend to be favored in the dynamics. The result is different, however, if we consider situations in which none of the equilibrium designs is Pareto superior. Let's consider, for instance, the following range of the parameters:

$$x > y + d, \quad d < (x - y)\xi, \quad (x - y)\delta < z < x - y - d \quad (15)$$

With reference to the payoffs in Table 1, the first three inequalities imply that the payoffs to t -agents are greater in $\{R^O, T^M\}$ than in $\{R^C, T^L\}$, whereas the payoffs to r -agents are greater in $\{R^C, T^L\}$ than in $\{R^O, T^M\}$, i.e. none of the designs is Pareto superior. The third inequality (together with the first), in line with Proposition 2, means instead that $\{R^O, T^M\}$ (or design 1) is more efficient than $\{R^C, T^L\}$ (or design 0), i.e. $\psi^1 > \psi^0$. Under these conditions, when the organization's state is in the basin of attraction of E_1 , the r -agents that are selected for idiosyncratic play have an incentive to vote for R^C because in so doing they may induce their best responding partners to vote T^L next period. For the same reason t -agents have an incentive to idiosyncratically vote for T^M when the organization's state is in the basin of attraction of E_0 . The combination of these effects may lead to the “tipping” of the group from one basin of attraction to the other.

The introduction of intentional idiosyncratic plays means that, independently of the initial conditions, each asymptotically stable state can be reached. This, however, does not mean that all states are equally likely. Intuition suggests that the voting equilibrium that requires a large amount of idiosyncratic plays to dislodge, while requiring little idiosyncratic plays to access, will tend to persist longer time than the other. At the same time, if dislodged, it will tend to reemerge readily. This is the equilibrium that is most likely to be associated with a persistent adoption of a system design.

In order to formalize the above intuition consider the following definitions:

Definition 4 (Young, 1998:105). Let r_{jk} , the reduced resistance on the path from E_j to E_k , be the minimal number of agents voting for E_j that, should they idiosyncratically switch their vote, would induce their best-responding partners to switch theirs. Then, $r_{01} = \min\{\rho^*, \omega^*\}$ and $r_{10} = \min\{1 - \rho^*, 1 - \omega^*\}$.

Definition 5 (Bowles, 2006:413). The stochastically stable equilibrium (SSE) is the one that occurs with non-negligible probability when the rate of idiosyncratic plays is arbitrarily small. In a 2×2 coordination game with two asymptotically stable equilibria E_j and E_k , E_j is SSE if and only if $r_{kj} < r_{jk}$.

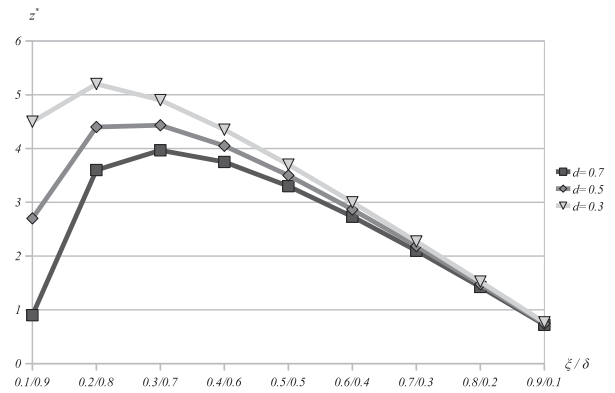


Fig. 3. Variation of z^* as a function of the ratio ξ/δ . Notice that for $z \leq z^*$ $\{R^O, T^M\}$ is SSE. The curves are drawn assuming $x=10$, $y=2$, and $n=1$. Three different values of d are considered: 0.7, 0.5 and 0.3. As the ratio n_i/n_r increases, z^* presents an inverted-U-shaped behavior, therefore making $\{R^O, T^M\}$ initially more and then less persistent for any given value of z and x .

Definitions 4 and 5 can be used in order to find the condition under which each of the two asymptotically stable voting equilibria identified in Proposition 3 qualifies as SSE. In particular, I obtain the following result:

Proposition 4. Suppose $\lambda > 0$. Then, in the dynamic system composed of Eqs. (13) and (14) there exist a $z^* = \delta(x - y - d/\xi)$ such that if $z < z^*$ E_1 is SSE.

Proof. See Appendix A. $\square$

The intuition behind Proposition 4 is straightforward. So long as the rents obtainable from the sale of proprietary software is sufficiently small (i.e. $z < z^*$), r -agents do not have much to lose in adopting design $\{R^O, T^M\}$ and therefore are more vulnerable to the idiosyncratic plays of t -agents. At the same time, when the transaction cost advantage (disadvantage) associated with an open-source regime is large (low) - notice that $\partial z^*/\partial x > 0$ and $\partial z^*/\partial y < 0$, t -agents strictly prefer design $\{R^O, T^M\}$ to $\{R^C, T^L\}$, and will try their best to get it. The combination of these two effects, makes design $\{R^O, T^M\}$ at the same time more likely to be adopted and less likely to be abandoned.

For any given value of z , x and y , the range of values for which $\{R^O, T^M\}$ is relatively persistent in the voting dynamics strictly depends on two additional factors: the design cost of modularity d and the relative sizes of the two groups δ and ξ . For what concerns d it is easily shown that $\partial z^*/\partial d < 0$, so that $\{R^O, T^M\}$ is more likely to be persistent the lower the value of d . This result could be used to interpret the impact of the so-called “digital revolution”: by enabling a greater modularization of software components, the advent of digital computing has dramatically reduced d and has therefore increased the rate of adoption of open-source productions. A similar point has been made also by Benkler (2002, 2006) and Baldwin and von Hippel (2011) among the others.

With respect instead to δ and ξ , their impact on the dynamics is less clearcut. In order to simplify the analysis, I report in Fig. 3 the value of z^* (vertical axis) for different combinations of δ and ξ , assuming $x=10$, $y=2$ and

normalizing the size of the group to one (i.e. $n = 1$).¹⁴ The horizontal axis reports the ratio $\xi/\delta = n_t/n_r$, and the curves are drawn for $d = 0.7, 0.5$, and 0.3 . The graph shows that, if we start from a situation in which the group is composed mainly by r -agents and we progressively increase the ratio n_t/n_r , z^* first increases—therefore making $\{R^O, T^M\}$ relatively more persistent for a given z , and then (after a certain threshold) decreases. The reasons behind such a behavior are related to the combination of two main forces that operate as the relative size of the two groups changes. The first force is associated with an effect on the value of the payoffs at the two equilibrium designs. As n_t increases and n_r decreases, in fact, the share of d that goes to each t -agent in $\{R^O, T^M\}$ diminishes, while the share of the benefits remains constant (n is fixed). This makes $\{R^O, T^M\}$ relatively more attractive. The same, obviously, holds for r -agents, since a reduction in n_r increases the share of the rents that would go to each of them in the case $\{R^C, T^L\}$ were to be adopted. For low value of n_t/n_r , however, the effect of an increase in n_t for t -agents is predominant over a reduction in n_r for r -agents, and the range of parameter for which $\{R^O, T^M\}$ is SSE enlarges (i.e. z^* increases).

After a certain threshold, however, this trend is inverted. As n_t and n_r begin to have similar dimensions, the reduction in the share of d which is due to an increase in n_t is more than offset by the increase in the share of z that follows a reduction in n_r , and z^* diminishes. In addition, when n_r becomes particularly small there is a second force that starts to play a role against the persistence of $\{R^O, T^M\}$, which is associated with an increase in the voting power of r -agents. For a given level of λ , in fact, the smaller n_r , the more r -agents can influence the voting process in favor of the equilibrium design that they prefer. When z is sufficiently large and n_r sufficiently small, such a preferred equilibrium is $\{R^C, T^L\}$. Notice that in this model such a power does not depend on their ability to coordinate on a common and advantageous position. Rather it is a consequence of the fact that, being small, they experience more “tipping” opportunities.

This inverted-U-shaped behavior of z^* with respect to the ratio n_t/n_r is partially related to the fact that, in this setting, the design of a modular code architecture assumes the characteristics of a quasi-public good. In order to have the degree of modularity which is necessary for open-source production to emerge, in fact, a big initial investment has to be put forward by one group (i.e. the t -agents) with the resulting benefits being shared by the organization as a whole.¹⁵ Referring to the literature on the theory of economic organizations (e.g. Alchian and Demsetz, 1972) this result could be interpreted as a version of the “ $1/n$ problem” where, as long as n_t is too small compared to n_r , the marginal cost of modularity is higher than its marginal benefit, therefore making design $\{R^O, T^M\}$ unsuitable.

In ultimate instance, Proposition 4, together with Proposition 2 and Definition 3 can be used in order to investigate the relationship between production efficiency and stochastic stability. The following proposition, in particular, summarizes the main finding:

Proposition 5. Suppose design $\{R^O, T^M\}$ is more efficient than $\{R^C, T^L\}$ in the sense of Definition 3 and no Pareto superior design exists. Assume also $x > y + d$. Then, there exist a range of values for the couple (z, d) such that E_0 (despite being inefficient) is SSE.

Proof. See Appendix A. $\square$

6. Discussion

Fig. 4 summarizes the main results of the model and suggests a useful way of interpreting the content of Proposition 5. The downward sloping 45° line defines the portions of the plane for which open-source production as opposed to close-source production is efficient: any point above the line identifies a combination of parameters (x, y, z, d) for which closed-source production is efficient, while any point below the line is a combination for which open-source production is efficient. The entire line represents instead z^* , so that for any point above such line closed-source production is SSE. z^* has been drawn by assuming $n_r < n_t$. As suggested in Proposition 5, so long as z^* has a vertical intercept which is lower than $x - y$, there exist a whole set of points (the shaded area in the graph) for which closed-source production is at the same time relatively inefficient and stochastically stable.

The fact that some organizations in the software industry effectively locate in this area of the graph is difficult to say theoretically, and it turns out to be mainly an empirical question. What Fig. 4 shows is the possibility that, under some plausible assumptions, relatively inefficient systems for software development can be persistently adopted. In this sense, Fig. 4 can be used in order to make sense of the process of systems differentiation that, following the development of digital technologies, has led to the high degree of organizational diversity that we presently observe.

Let's consider for instance points A and B in the graph, which represent a combination of parameters characterized by given rents (higher in A than in B) and high design costs of modularity (the same for both). These points could be interpreted as two pieces of software that were developed prior to the so-called digital revolution, let's say in the late '80s, by two distinct companies operating in the industry. At that time technology made it very expensive to design modular platforms (relatively high d), so that closed-source production tended to be the only viable and efficient solution. The economy, as a consequence, exhibited a relatively homogeneous nature in terms of organizational demography, with close-source production being the prevailing form (both A and B fall, in fact, in this category).

After the technological shock represented by the widespread diffusion of the Internet and the huge fragmentation of computational capabilities, however, the design cost of modularity has sensibly dropped. At the same time, in the domain of property rights, the diffusion of FOSS

¹⁴ These values are chosen for convenience in such a way that the effects are expressed in the right order of magnitude. The general results, however, do not depend on them.

¹⁵ The author is grateful to Ugo Pagano for the idea of modularity as a quasi-public good.

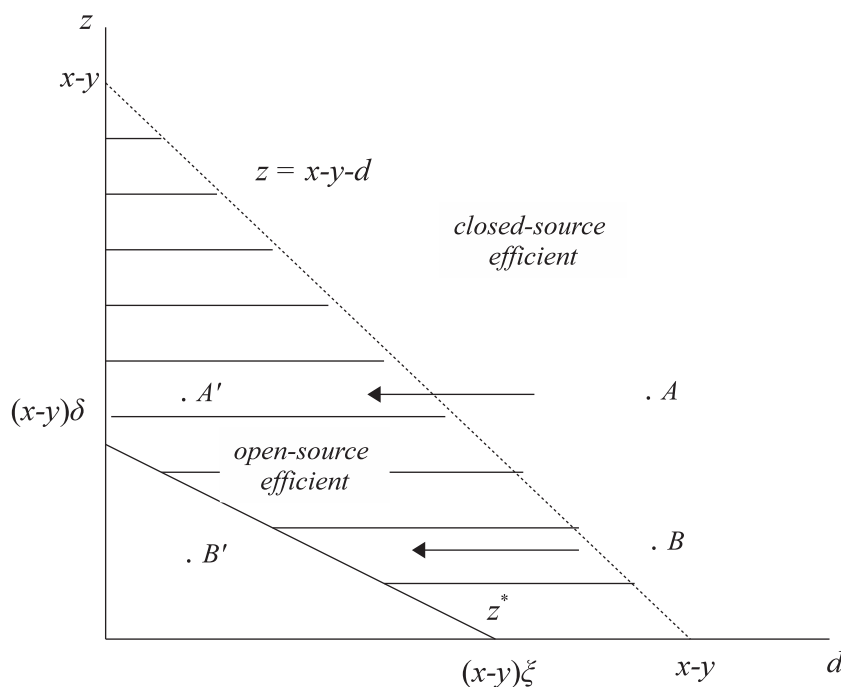


Fig. 4. Production efficiency and stochastic stability. Note: the shaded area reflects the set of points for which closed-source production is at the same time inefficient and stochastically stable. The arrows indicate the shift caused by the diffusion of digital technologies. Starting from an economy mainly populated by closed-source productions, the reduction in the design cost of modularity brought about the emergence of an ecology of organizational forms in which open- and closed-source production co-exist.

licenses had only a negligible effect on the rents that could be earned from proprietary software, so that the latter remained close to their initial level. As a consequence the systems of production experienced a horizontal shift in the (d, z) space as the one depicted in the figure ($A \rightarrow A'$ and $B \rightarrow B'$). In the cases in which the rents prior to the drop in d were low, point B in the graph, the organizations most likely experienced a change from closed- to open-source production (point B') which had become, by that time, relatively efficient and profitable (e.g. Netscape Communicator 4.0 turned into Mozilla Firefox). On the contrary, when the rents before the reduction in d were high, point A , the organization still had an incentive in maintaining a closed-source type of production (point A') despite the fact that, under the new technological environment, it had become relatively inefficient (e.g. Internet Explorer). As a result, in analogy with the process of biological speciation, distinct trajectories of organizational change have gradually emerged. The organizations that, in because of low expected rents, had little to lose by switching to open-source production made the move, while the others kept the design associated with the old system. Such a divergence has in turn led to a variegated ecology of organizational forms in which open- and closed-source productions have increasingly co-existed, *independently* of their relative (in)efficiency. As long as each form was able to sustain a sufficiently high degree of economic performance (e.g. through the development of different revenue generation models), no selective pressure could readily wipe out the relatively unfit systems and organizational diversity could persist.

Overall, the model offers one possible explanation for the reason why we observe both open- and closed-source productions in the software industry. In light of the obtained results, some additional research should be carried out in order to test its validity. In particular, the attention should be placed on the hypothesis of persistent systemic inefficiency. If such an hypothesis were to be verified, in fact, there could be room to argue in favor of government interventions in order to restore efficiency in the economy. In this sense, policy interventions could go along two main lines: (i) a reform of the intellectual property rights law aimed at reducing rents from software sales; and (ii) a deeper involvement of public institutions (e.g. Universities) in the design of modular platforms with the objective of reducing the 'under-supply' of modularity which follows from its own quasi-public good nature.

As a concluding remark let me highlight some limitations of the paper. Apart from the already mentioned degree of abstractness, one important limit is that there is no direct account of the competitive interaction between open- and closed-source productions. In this sense, while offering an explanation for the emergence of organizational diversity, the paper does not study if and for how long such diversity is effectively going to last. Several factors could play an important role in these terms, including the actual frequency of the two systems in the overall economy and the time span over which the selection of the fittest can take place. A careful analysis of such factors, however, requires a completely different approach to the issue and is thus left for future research.

Acknowledgments

This paper was completed while the author was visiting the Berkman Center for Internet and Society at Harvard University. The author is grateful to Ugo Pagano, Sam Bowles, Yochai Benkler, two anonymous referees and conference participants at the Sant'Anna School of Advanced Studies, the Scientific Institute of Cargese and the Politecnico di Milano for their useful comments. The usual caveat applies.

Appendix A.

Proof of Proposition 1. Let's consider $Q > 0, z > 0, l = m > 0, x > 0$ and $y > 0$. (a) $\{R^C, T^L\}$ is proven to be a Nash equilibria as long as: (i) $[Q - (l - y)]/n + z/n_r \geq (Q - l)/n$, and (ii) $[Q - (l - y)]/n \geq (Q - m)/n - d/n_t$. Both conditions are self-explained. (b) For $\{R^C, T^M\}$ and $\{R^O, T^L\}$ to be Nash equilibria two necessary conditions are that T^M is a best response to R^C and R^O is a best response to T^L . These conditions, however, can never be verified because otherwise they would violate point (a) above. (c) $\{R^O, T^M\}$ is proven to be a Nash equilibria as long as: (i) $[Q - (m - x)]/n \geq (Q - m)/n + z/n_r$, and (ii) $[Q - (m - x)]/n - d/n_t \geq (Q - l)/n$. Simple algebra shows that the latter conditions reduce to $x \geq z(n/n_r) = z/\delta$ and $d \leq x(n_t/n) = x\xi$.

Proof of Proposition 2. (a) Let's call $\{R^C, T^L\}$ design 0 (benchmark) and $\{R^O, T^M\}$ design 1. From Definition 3 it follows that:

$$\psi^0 = \left[\frac{Q - (l - y)}{n} + \frac{z}{n_r} \right] n_r + \left[\frac{Q - (l - y)}{n} \right] n_t \quad (\text{A.1})$$

$$\psi^1 = \left[\frac{Q - (m - x)}{n} \right] n_r + \left[\frac{Q - (m - x)}{n} - \frac{d}{n_t} \right] n_t \quad (\text{A.2})$$

Simple algebra shows that $\psi^1 > \psi^0$ if and only if:

$$z < x - y - d \quad (\text{A.3})$$

Since $z \geq 0$ by assumption, $x \geq y + d$ is a necessary condition for inequality A.3 to hold. (b) Two necessary and sufficient conditions for $\{R^O, T^M\}$ to be Pareto superior are that: $[Q - (m - x)]/n \geq [Q - (l - y)]/n + z/n_r$ and $[Q - (m - x)]/n - d/n_t \geq [Q - (l - y)]/n$. After some simple algebra such conditions reduces to:

$$z \geq (x - y)\delta \quad d \leq (x - y)\xi \quad (\text{A.4})$$

Since both $z \geq 0$ and $d \geq 0$ by assumption, $x \geq y$ is a necessary condition for inequalities (A.4) to hold.

Proof of Proposition 3. The five voting equilibria are derived by simply solving the system (13)–(14) for $\omega_\tau = 0$ and $\rho_\tau = 0$. The proof in this case is omitted. The asymptotic properties of each equilibrium are derived by analyzing the Jacobean Matrix J associated to system (13)–(14):

- At $\{0, 0\}$, $\text{Tr}(J) = -\beta/n(2y + z/\delta + d/\xi) < 0$, $\text{Det}(J) = \beta^2(y/n + z/n_r)(y/n + d/n_t) > 0$. Hence, $\{0, 0\}$ is asymptotically stable.
- At $\{1, 0\}$, $\text{Tr}(J) = \beta/n(y + z/\delta + x - d/\xi) > 0$ for any $d < x\xi$, $\text{Det}(J) = \beta^2(y/n + z/n_r)(x/n - d/n_t) > 0$ for any $d < x\xi$. Hence, $\{1, 0\}$ is unstable.

- At $\{0, 1\}$, $\text{Tr}(J) = \beta/n(x - z/\delta + y + d/\xi) > 0$ for any $x > z/\delta$, $\text{Det}(J) = \beta^2(x/n - z/n_r)(y/n + d/n_t) > 0$ for any $x > z/\delta$. Hence, $\{0, 1\}$ is unstable.
- At $\{1, 1\}$, $\text{Tr}(J) = \beta(z/n_r - x/n + d/n_t - x/n) < 0$ for any $x > z/\delta$ and $d < x\xi$, $\text{Det}(J) = \beta^2(z/n_r - x/n)(d/n_t - x/n) > 0$ for any $x > z/\delta$ and $d < x\xi$. Hence, $\{1, 1\}$ is asymptotically stable.
- At $\{\rho^*, \omega^*\}$, $\text{Det}(J) = -\beta^2(y + d/\xi)(x - d/\xi)(y + z/\delta)(x - z/\delta)/n^2(x + y)^2 < 0$ for any $x > z/\delta$ and $d < x\xi$. Hence, $\{\rho^*, \omega^*\}$ is a saddle.

Proof of Proposition 4. From Definition 4 and the value of ρ^* and ω^* reported in Proposition 3 it follows that:

$$r_{01} = \omega^* = \frac{y + z/\delta}{x + y} \quad \text{and} \quad r_{10} = 1 - \rho^* = \frac{x - d/\xi}{x + y} \\ \Leftrightarrow \quad z\xi \leq d\delta \quad (\text{A.5})$$

$$r_{01} = \rho^* = \frac{y + d/\xi}{x + y} \quad \text{and} \quad r_{10} = 1 - \omega^* = \frac{x - z/\delta}{x + y} \\ \Leftrightarrow \quad z\xi > d\delta \quad (\text{A.6})$$

According to Definition 5, E_1 is SSE if and only if $r_{01} < r_{10}$. Simple algebra shows that, given Eqs. A.5 and A.6, the latter condition holds if and only if $z < \delta(x - y - d/\xi) = z^*$.

Proof of Proposition 5. From Proposition 4 it follows that E_0 is SSE if and only if:

$$z > \delta(x - y - d/\xi) = z^* \quad (\text{A.7})$$

Considering Eq. (A.7) jointly with Proposition 2, E_0 can be SSE while being relatively inefficient as long as the following holds:

$$\delta(x - y - d/\xi) < z < x - y - d \quad (\text{A.8})$$

For $d = 0$, the closed interval defined by (A.8) always exists, since $\delta < 1$ by definition. This, together with the fact that $\partial z^*/\partial d < 0$, proves the proposition.

References

- Alchian, A.A., Demsetz, H., 1972. Production, information costs, and economic organization. *The American Economic Review* 62 (5), 777–795.
- Aoki, M., 1998. The evolution of organizational conventions and gains from diversity. *Industrial and Corporate Change* 7 (3), 399–431.
- Aoki, M., 2001. *Toward a Comparative Institutional Analysis*. MIT Press, Cambridge.
- Baldwin, C.Y., Clark, K.B., 2000. *Design Rules, Vol. 1: The Power of Modularity*. Cambridge, MIT Press.
- Baldwin, C.Y., Clark, K.B., 2006. The architecture of participation: does code architecture mitigate free riding in the open source development model? *Management Science* 52, 1116–1127.
- Baldwin, C.Y., von Hippel, E., 2011. Modeling a paradigm shift: from producer innovation to user and open collaborative innovation. *Organization Science* 22, 1399–1417.
- Benkler, Y., 2002. Coase's penguin, or linux and the nature of the firm. *The Yale Law Journal* 112 (3), 369–446.
- Benkler, Y., 2006. *The Wealth of Networks: How Social Production Transforms Markets and Freedom*.
- Benkler, Y., 2010. Law, policy and cooperation. In: Balleisen, E., Moss, D. (Eds.), *Government and Markets: Towards a New Theory of Regulation*. Cambridge University Press.
- Bonaccorsi, A., Rossi, C., 2003. Why open source software can succeed. *Research Policy* 23 (7), 1243–1258.
- Bowles, S., 1985. The production process in a competitive economy: Walrasian, neo-hobbesian, and marxian. *The American Economic Review* 75 (1), 16–36.

- Bowles, S., 2006. *Microeconomics: Behavior, Institutions, and Evolutions*. Princeton University Press, Princeton.
- David, P.D., Rullani, F., 2008. Dynamics of innovation in an open source collaboration environment: lurking, laboring, and launching floss projects on sourceforge. *Industrial and Corporate Change* 17 (4), 647–710.
- David, P.D., Shapiro, J.S., 2008. Community-based production of open-source software: what do we know about the developers who participate? *Information Economics and Policy* 20, 364–398.
- den Besten, M., Dalle, J.-M., Galia, F., 2008. The allocation of collaborative efforts in open-source software. *Information Economics and Policy* 20, 316–322.
- Elkin-Loren, N., Salzberger, E.M., 2000. Law and economics in cyberspace. *International Review of Law and Economics* 19, 553–581.
- Gambardella, A., Hall, B.H., 2006. Proprietary vs. public domain licensing of software and research products. *Research Policy* 35 (6), 875–892.
- Gintis, H., 2009. *Game Theory Evolving: A Problem Centered Introduction to Modeling Strategic Interaction*, 2nd edition. Princeton University Press, Princeton.
- Johnson, J.P., 2006. Collaboration, peer review and open source software. *Information Economics and Policy* 18 (4), 477–497.
- Kahneman, D., 2003. Maps of bounded rationality: psychology for behavioral economics. *The American Economic Review* 93 (5), 1449–1475.
- Lakhani, K.R., von Hippel, E., 2003. How open source software works: free user-to-user assistance. *Research Policy* 32 (6), 923–943.
- Lakhani, K.R., Wolf, R.G., 2005. Why hackers do what they do: understanding motivations and effort in free/open source software projects. In: Feller, J., Fitzgerald, B., Hassam, S., Lakhani, K.R. (Eds.), *Perspectives on Free and Open Source Software*. MIT Press.
- Langlois, R.E., Garzarelli, G., 2008. Of hackers and hairdressers: modularity and the organizational economics of open-source collaboration. *Industry and Innovation* 15 (2), 125–143.
- Lerner, J., Tirole, J., 2002. Some simple economics of open source. *The Journal of Industrial Economics* 50 (2), 197–234.
- Lerner, J., Tirole, J., 2005. The economics of technology sharing: open source and beyond. *The Journal of Economic Perspectives* 19 (2), 99–120.
- Mac Cormack, A., Rusnak, J., Baldwin, C.Y., 2006. Exploring the structure of complex software designs: an empirical study of open source and proprietary code. *Management Science* 52 (7), 1015–1030.
- Marglin, S.A., 1974. What do bosses do? The origins and functions of hierarchy in capitalist production. *Review of Radical Political Economy* 6, 60–112.
- McGowan, D., 2001. Legal implications of open-source software. *University of Illinois Legal Review* 2001 (1), 241–304.
- Naidu, S., Hwang, S.H., Bowles, S., 2010. Evolutionary bargaining with intentional idiosyncratic play. *Economic Letters* 109 (1), 31–33.
- Nelson, R., Winter, S.G., 1982. *An Evolutionary Theory of Economic Change*. Harvard University Press, Cambridge.
- Pagano, U., 1993. Organizational equilibria and institutional stability. In: Bowles, S., Gintis, H., Gustafsson, B. (Eds.), *Markets and Democracy: Participation, Accountability and Efficiency*. Cambridge University Press, Cambridge.
- Pagano, U., 2001. The origin of organizational species. In: *The Evolution of Economic Diversity*, Routledge, London.
- Pagano, U., 2011. Interlocking complementarities and institutional change. *Journal of Institutional Economics* 7 (3), 373–392.
- Pagano, U., Nicita, A., 2001. *The Evolution of Economic Diversity*. Routledge, London.
- Pagano, U., Rossi, M.A., 2004. Incomplete contracts, intellectual property and institutional complementarities. *European Journal of Law and Economics* 18 (1), 55–67.
- Pagano, U., Rowthorn, R., 1994. Ownership, technology and institutional stability. *Structural Change and Economic Dynamics* 5 (2), 221–242.
- Parnas, D.L., 1972. On the criteria to be used in decomposing systems into modules. *Communications of the ACM* 15, 1053–1058, December.
- Putterman, L., 1982. Some behavioral perspective on the dominance of hierarchical over democratic forms of enterprise. *Journal of Economic Behavior and Organization* 3 (2/3), 139–160.
- Shah, S., 2006. Motivation, governance, and the viability of hybrids forms in open source software development. *Management Science* 52 (7), 1000–1014.
- Simon, H.A., 1962. The architecture of complexity. *Proceedings of the American Philosophical Society* 106, 467–482.
- von Hippel, E., 2007. Horizontal innovation networks—by and for users. *Industrial and Corporate Change* 16 (2), 293–315.
- von Hippel, E., von Krogh, G., 2003. Open source software and the private-collective innovation model: issues for organization science. *Organization Science* 14 (2), 209–223.
- West, J., O'Mahony, S., 2008. The role of participation architecture in growing sponsored open source communities. *Industry and Innovation* 15 (2), 145–168.
- Williamson, O.E., 1985. *The Economic Institutions of Capitalism*. The Free Press, New York.
- Young, P.H., 1998. *Individual Strategy and Social Structure: An Evolutionary Theory of Institutions*. Princeton University Press, Princeton.